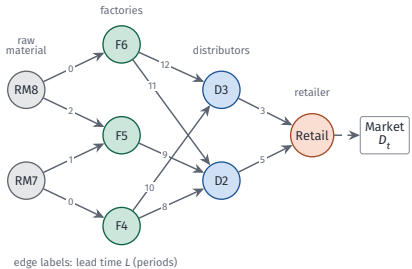
gym-invmgmt

Benchmarking optimization, learning and LLM agents for inventory control

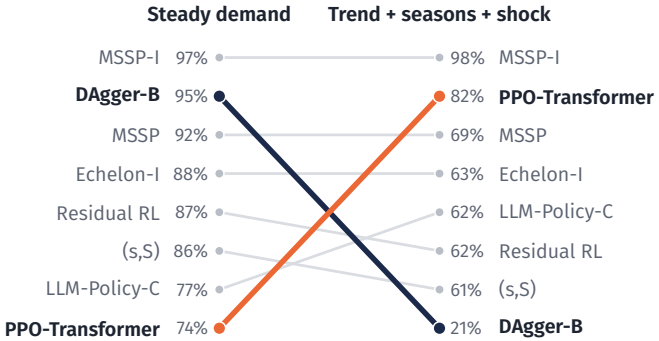
Reza Barati Qinmin Vivian Hu

Department of Computer Science, Toronto Metropolitan University

Computer Science Weekly Seminar



Change one assumption, and the ranking flips



Eight of the 18 controllers in our main results table, one simulator, one metric (% of an Oracle's profit). Same networks and shortage rules: **only the demand process changes**.

DAgger-B: 2nd → last.

PPO-Transformer: last → 2nd.

Among all 18: 2nd → 17th and 15th → 2nd.

By the end of the talk, I hope you'll agree that "which method is best?" needs a contract attached.

Means over 4 + 4 matched core scenarios (both networks, backlog and lost sales, goodwill off), released results; ranks among these 8.

Plan for today

1. **The problem.** Why inventory control is a good, and slightly underrated, testbed for sequential decision-making.
2. **The benchmark.** One environment contract that optimizers, rules, neural policies and LLMs can all be evaluated under.
3. **The experiments.** About thirty controllers on 22 stress scenarios: who wins, where, and at what cost.
4. **Lessons and open problems.** What carries over to benchmarking in general, and what is still unsolved.

Part 1

The problem

1 The problem

2 The benchmark

3 The experiments

4 Lessons and open problems

A one-period puzzle

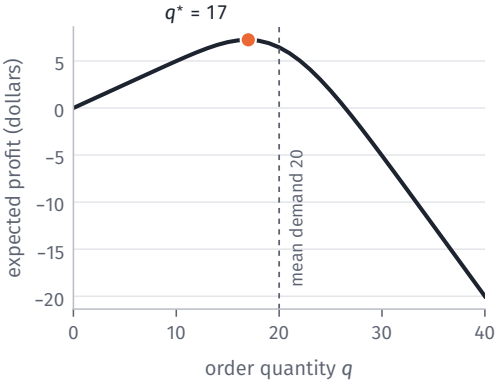
Demand averages **20** units (Poisson). You sell at \$2.00 and buy at \$1.50; leftovers are worthless.

How many do you order?

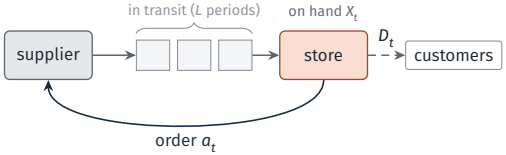
Running out costs \$0.50 of margin; overstocking costs \$1.50.

Critical ratio $\frac{p - c}{p} = 0.25$, so order the 25th percentile of demand: $q^* = 17$.

A forecast is not a decision.



Classical answer: order up to a demand quantile

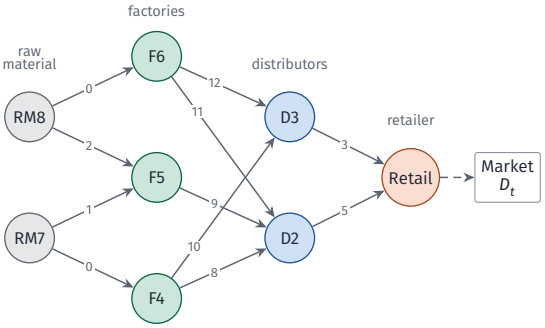


- Each period: look at stock on hand and in transit, then place an order a_t.
- The order arrives L periods later; demand D_t is random.
- Too much stock costs holding; too little costs lost sales or backlog penalties.
- Classical answer (one item, stationary demand, linear costs, backorders): *order up to* a demand quantile over the lead time plus one review period,

$$S^* = F^{-1}_{D_{L+1}}\left(\frac{b}{b+h}\right), \qquad a_t = \max(0, S^* - \mathrm{IP}_t),$$

  b = shortage cost, h = holding cost, IP = on hand + in transit − backlog.

Real supply chains are networks



edge labels: lead time L (periods)

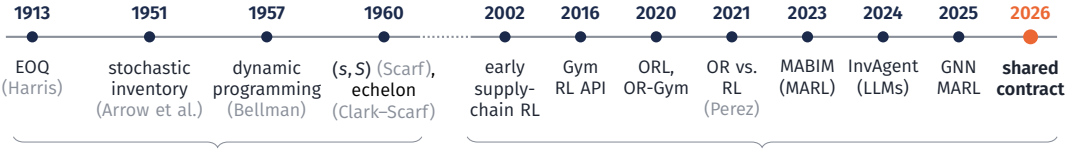
- 8 sites in 4 echelons plus the market; 11 reorder links.
- Lead times from 0 to 12 periods; factories have capacity limits.
- One controller sets all 11 orders every period, $a_t \in \mathbb{R}^{11}_{\geq 0}$, from 71 numbers, 61 of them orders still in transit.
- The second topology is a serial chain.

Default network and all its parameters: OR-Gym (Hubbs et al. 2020), unchanged; also used by Perez et al. (2021).

Delays, coupling and shifting demand break the textbook answer

- **Delayed effects.** An order placed now may land 12 periods later, so the state must carry the whole pipeline.
- **Coupling.** A distributor can only ship what its factories hold; requested and executed orders differ.
- **Bullwhip risk.** Order swings *can* amplify as they move upstream, depending on the policy.
- **Non-stationary demand.** Trends, seasons, shocks; real sales traces are messier still.
- **Feedback** (optional goodwill). Poor service today can shrink demand tomorrow.
- **Scale.** A full 3-branch scenario tree over 30 periods has $3^{30} \approx 2 \times 10^{14}$ paths, so solvers sample instead; the state is continuous.

A century of methods, a decade of benchmarks



OR models behind our baselines. The Oracle and DLP solve linear programs; MSSP is a stochastic program solved by sample-average approximation and re-planned every period.

Shared APIs and benchmarks made runs repeatable, but studies still differ in topology, demand, information access, constraints and accounting.

Same problem, different contracts

Is an RL policy better than a stochastic program? It depends on choices that differ from study to study:

- network topology
- demand regime
- **information access**: does the agent see the demand model?
- feasibility and action bounds
- backlog vs. lost sales
- KPI definitions and cost accounting
- training and online compute budget

An ML analogy

Comparing classifiers that were each trained and tested on a different split, with a different metric, and then ranking them by the reported number.

Same MSSP solver, blind vs. informed: **65.7%** vs. **94.9%** of the Oracle.

Representative benchmarks each cover a different slice

Benchmark / study	Control	Demand	Observations	Methods compared	Goodwill
ORL (Balaji et al. 2020)	central	stationary Poisson	flat vector	PPO, DQN, heuristics	–
OR-Gym (Hubbs et al. 2020)	central	stationary	flat vector	PPO, heuristics, MIP	–
Perez et al. (2021)	central	stationary	flat vector	DLP/MSSP, PPO, Oracle	–
MABIM (Yang et al. 2023)	decentral.	stat./non-stat.	SKU vector	IPPO, QTRAN, base-stock	–
InvAgent (Quan & Liu 2024)	decentral.	multi-regime	text prompt	GPT-4 agents, IPPO/MAPPO	–
GNN-MARL (Kotecha & del Rio Chanona 2025)	CTDE	stoch./shock	network graph	GNN-MAPPO, heuristics	–
gym-invmgmt (2026)	central	stat., non-stat., M5 trace	vector, graph, sequence, text	OR, heuristics, RL, imitation, hybrid, LLM	✓

OR-Gym and Perez et al. are our direct lineage. The MARL and LLM lines answer a different question (decentralized coordination).

Six of the twelve studies in our Table 1, each in its primary setting. Goodwill: realized service changes future demand.

Contributions

1 A benchmark contract

Configurable DAG topologies, stochastic and empirical demand, backlog or lost sales, endogenous goodwill; vector, graph, sequence and text observations.

2 A released scenario matrix

22 core scenarios × 10 evaluation seeds; 28 controller configurations and an LLM baseline, all scored against an Oracle reference.

3 Generalist evaluation

PPO, SAC, Transformer, GNN, residual and imitation checkpoints, one per topology, tested without per-scenario retraining; a separate zero-shot graph-transfer test.

4 Cross-paradigm findings

Informed stochastic programming leads; information helps planners most; learned, LLM and transfer tests expose failure modes.

Open: environment on PyPI; agents, scenarios, results and checkpoints on GitHub and Hugging Face.

Part 2

The benchmark

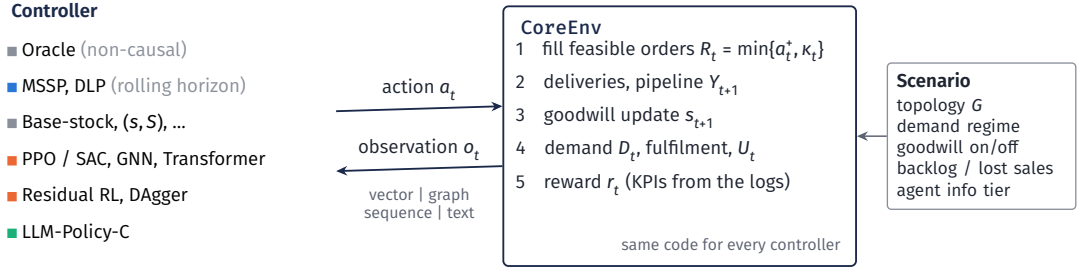
1 The problem

2 The benchmark

3 The experiments

4 Lessons and open problems

One contract, many controllers



Observation wrappers change the encoding, never the physics. The scenario block is the only thing that varies between runs.

What we reused, what we changed, what is new

Reused unchanged (OR-Gym)

- default 9-node network with all its parameters
- event order: order, deliver, sell, book profit
- profit = revenue − purchasing − operating − holding − shortage
- 30-period episodes; backlog or lost sales

Changed

- Gymnasium API; NumPy state
- continuous orders (OR-Gym rounds to integers)
- shared stock and capacity allocated link by link
- observation: last realized demand, arrival-indexed pipeline
- serial chain: same format, new parameters

New

- demand engine (trend, season, shock, M5) and goodwill
- YAML topologies; graph, token, per-link, text interfaces
- blind / informed tiers, 22-scenario matrix, generalist protocol
- Oracle under goodwill; Residual RL, LLM-Policy-C
- released results, checkpoints, evaluator

OR-Gym: Hubbs et al. (2020); the same network is used by Perez et al. (2021). The compared methods (PPO, SAC, DAgger, MSSP, DLP, base-stock rules) come from the literature; the implementations are ours.

Agents request; the simulator ships what is feasible

Fixed event order

1. allocate raw requests into feasible filled orders
2. advance lead-time deliveries and pipeline
3. update goodwill from last period's unmet demand
4. realize demand, record fulfilment
5. compute the reward (KPIs are computed from the logs after the episode)

Requested vs. filled

$$R_t^{(e)} = \min\left\{\max(a_t^{(e)}, 0),\ \kappa_t^{(e)}\right\}$$

κ: what upstream stock and factory capacity allow.

Conservation

$$X_{t+1}^{(j)} = X_t^{(j)} + \sum_{i \in \mathrm{pred}(j)} R_{t-L_{ij}}^{(i,j)} - \frac{1}{v_j} \sum_{k \in \mathrm{succ}(j)} S_t^{(j,k)}$$

$$Y_{t+1}^{(e)} = Y_t^{(e)} - R_{t-L_e}^{(e)} + R_t^{(e)}$$

The pipeline is rebuilt from *filled* orders, so agents never observe "phantom" stock that was requested but never shipped.

Reward is plain profit; scores are % of Oracle profit

$$r_t = \alpha^t \sum_{j \in M} \Big[\underbrace{\textstyle\sum_k p_{jk} S_t^{(j,k)}}_{\text{revenue}} - \underbrace{\textstyle\sum_i p_{ij} R_t^{(i,j)}}_{\text{procurement}} - \underbrace{h_j X_{t+1}^{(j)}}_{\text{holding}} - \underbrace{\textstyle\sum_i g_{ij} Y_{t+1}^{(i,j)}}_{\text{pipeline}} - \underbrace{\tfrac{o_j}{v_j} \textstyle\sum_k S_t^{(j,k)}}_{\text{operating}} - \underbrace{\textstyle\sum_k b_{jk} U_t^{(j,k)}}_{\text{shortage}} - \underbrace{\textstyle\sum_i K_{ij} \mathbb{1}\{R_t^{(i,j)} > 0\}}_{\text{fixed}} \Big]$$

- Holding and pipeline costs on end-of-period stock; shortage on unmet demand.
- The fixed-cost and discount terms are switched off in every run ($K = 0$, $\alpha = 1$).
- Every term is exported as a KPI, with fill rate, service level and a bullwhip ratio.
- Main metric: **% of Oracle profit**, per scenario, then averaged.

Same state, several interfaces; three information levels

Observation interfaces

vector (71 + 58 features)	PPO-MLP, SAC
+ graph built by the policy	PPO-GNN, DAgger, GNN-IL
per-link features	Residual RL
node tokens	PPO-Transformer
4-frame temporal stack	ST-PPO
text prompt (network size)	LLM-Policy-C
simulator state, read directly	heuristics, OR, Oracle

Encodings differ; transition code does not.

Information protocol

- **Blind**: realized demand history, pipeline and inventory.
- **Informed**: also given the demand model: the current mean $\mu(t)$ for learned agents, the mean path over the horizon for solvers and most heuristics. Causal, except on the 2 M5 rows, where the mean path is the replayed trace.
- **Oracle**: non-causal reference that sees the future demand path.

Information access is reported *separately* from architecture, so an informed model is not mistaken for a better one.

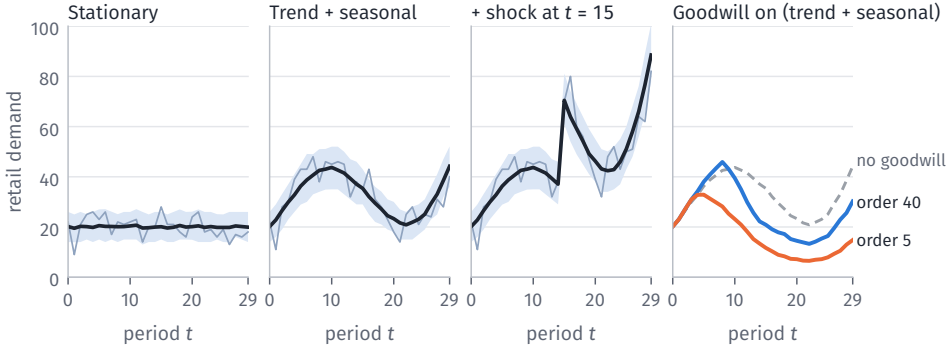
Demand is composable: trend, season, shock, real traces

- Base: stationary Poisson (μ = 20).
- Composable effects: linear trend, sinusoidal seasonality, step shocks, noise scaling.
- External traces: an M5 (Walmart) item-store window with high rolling CV, rescaled to mean 20 and replayed deterministically.
- Optional endogenous goodwill (two slides on).

M5 drives terminal demand only; lead times, capacities and costs stay controlled.

```
env = gym.make(
    "GymInvMgmt/MultiEchelon-v0",
    demand_config={
        "type": "trend+seasonal+shock",
        "base_mu": 20,
        "use_goodwill": True,
        "shock_time": 15,
        "shock_mag": 2.0,
    },
    backlog=False,
)
```

The simulator in action



Mean (black), 10–90% band and one sample path, 200 seeds. With goodwill on (right), a lean policy shrinks its own market to about a third of the exogenous demand.

gym-invmgmt 0.2.1 from PyPI, default network. Constant-order policies, for illustration only (not benchmark agents).

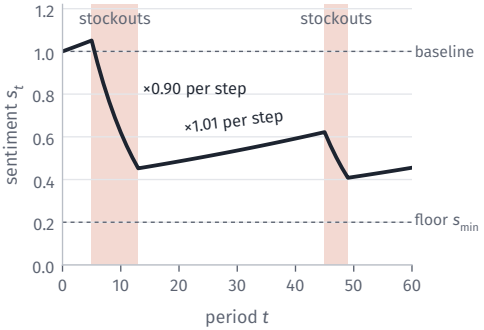
Stockouts shrink future demand, and recovery is slow

$$s_{t+1} = \begin{cases} \max(s_{\min},\ \gamma_{\text{drop}}\, s_t) & \text{stockout at } t \\ \min(s_{\max},\ \gamma_{\text{rec}}\, s_t) & \text{otherwise} \end{cases}$$

$$\lambda_{t+1} = s_{t+1}\, \bar{\lambda}_{t+1}$$

with $\gamma_{\text{drop}} = 0.90$, $\gamma_{\text{rec}} = 1.01$, $s_{\min} = 0.2$, $s_{\max} = 2.0$.

- Unmet demand lowers sentiment, which scales the next demand mean.
- Same exogenous demand, different future depending on the policy's own past failures.
- Hard for rolling-horizon solvers too: future demand is policy-dependent.



Goodwill recovers only if over 91% of periods are stockout-free

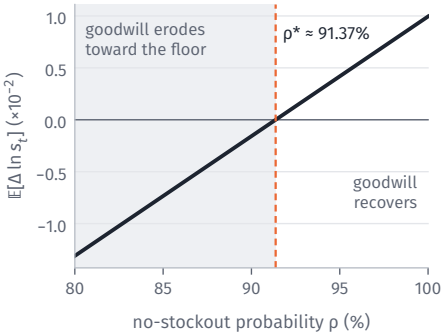
Proposition. Let ρ be a stationary no-stockout probability. Ignoring the floor and cap, $\mathbb{E}[\Delta \ln s_t] \geq 0$ iff

$$\rho \;\geq\; \rho^{*} = \frac{\ln(1/\gamma_{\text{drop}})}{\ln(1/\gamma_{\text{drop}}) + \ln \gamma_{\text{rec}}} \approx 91.37\%.$$

Sketch. $\Delta \ln s_t$ is $\ln \gamma_{\text{rec}}$ w.p. ρ and $\ln \gamma_{\text{drop}}$ w.p. $1 - \rho$; set the mean to zero.

One stockout period takes about $\ln(1/0.90)/\ln 1.01 \approx 10.6$ clean periods to undo.

ρ is the benchmark's service-level KPI (share of periods with no unmet demand), not the fill rate. At $\rho = 90\%$ the market still shrinks.



Standard Gymnasium: install, make, step

```
# pip install gym-invmgmt
import gymnasium as gym
import gym_invmgmt

env = gym.make("GymInvMgmt/MultiEchelon-v0")
obs, info = env.reset(seed=42)

done = False
while not done:
    action = env.action_space.sample()
    obs, reward, terminated, truncated, info = \
        env.step(action)
    done = terminated or truncated
```

Try it in Colab

Runs the environment, a base-stock policy and a comparison with the published baselines in about five minutes on a CPU.

`MultiEchelon-v0`: 11 actions, 71-dim observation. `Serial-v0`: 3 actions, 15-dim. Custom networks from YAML; wrappers for integer orders, episode logging, domain randomization, and graph and temporal views.

Part 3

The experiments

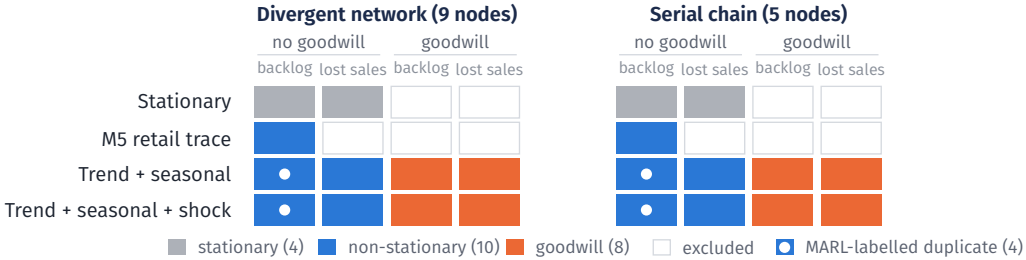
1 The problem

2 The benchmark

3 The experiments

4 Lessons and open problems

22 core scenarios, 10 seeds each



2 × 4 × 2 × 2 = 32 cells. Goodwill is not paired with stationary or M5 demand (it targets service-dependent non-stationarity), and M5 runs with backlog only (observed sales are censored).

Metric: scenario-wise % of Oracle profit, averaged; plus online seconds per episode.

Six families, about thirty controllers

Family	Agents	How it decides
Reference	Oracle	clairvoyant LP (PuLP + CBC); fixed-point procedure under goodwill
Rolling-horizon OR	MSSP, MSSP-I, DLP, DLP-I	sample-average multi-stage stochastic program; expected-value LP; re-solved online
Heuristics	Newsvendor, (s, S), ExpSmooth, Echelon (+ informed)	critical ratio, reorder point / order-up-to, Holt smoothing, Clark–Scarf-style echelon base-stock
Imitation	DAgger-B, DAgger-G, GNN-IL	dataset aggregation on GNN policies; behavioural cloning + PPO fine-tuning
Deep RL	PPO-MLP, PPO-GNN, PPO-Transformer, ST-PPO, SAC	PPO with MLP, directed MPNN, node-token or node×time-token Transformer; soft actor-critic
Hybrid	Residual RL	PPO scales the newsvendor order by $1 + \delta$, $\lvert\delta\rvert \le 0.5$
Foundation	LLM-Policy-C	Qwen2.5-1.5B picks six base-stock parameters once per episode

28 controller configurations (22 primary, 6 ablations) and the LLM baseline, scored against the Oracle.

Training protocol: generalists, not specialists

- RL agents: one checkpoint per **architecture × topology**, trained with Stable-Baselines3 / PyTorch.
- Randomized at every reset: trend, season, shock (each on or off), goodwill, backlog vs. lost sales, noise scale; never the real M5 trace.
- Imitation agents are outside this pool: GNN-IL learns from one stationary scenario.
- Every agent: 22 scenarios × 10 seeds = 220 episodes, no retraining.

A deliberate "generalization tax"

OR methods re-plan online from the current state (and, if informed, the current demand parameters). Learned policies must cover every regime with one set of weights.

So the learned numbers measure how one model covers the regimes it was trained on (the real M5 trace aside), not scenario-tuned optimality.

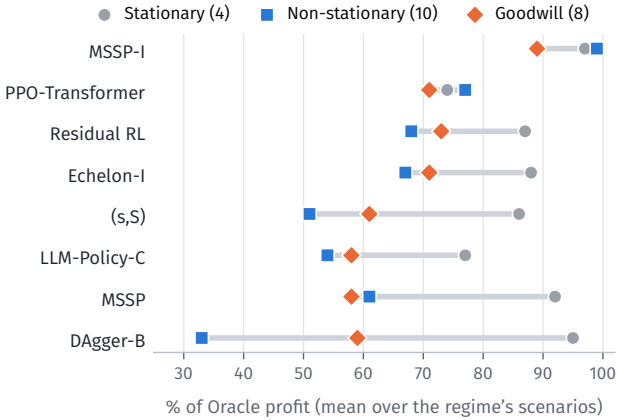
MSSP-I leads in every regime

	Stationary (4)	Non-stat. (10)	Goodwill (8)	All (22)	s / ep
MSSP-I	**97**	**99**	**89**	**95**	7.68
MSSP	92	61	58	66	10.28
DLP-I	80	50	58	58	1.00
DLP	78	36	52	49	1.06
Echelon-I	88	67	71	72	0.054
(s,S)	86	51	61	61	0.032
Echelon	84	50	55	58	0.058
Newsvendor	83	46	56	56	0.049
ExpSmooth	84	45	55	56	0.037
DAgger-B	95*	33	59	53	0.273
GNN-IL	84	3	25	26	0.275
PPO-Transformer	74	77*	71	75*	0.229
ST-PPO	69	59	64	63	0.269
PPO-MLP	77	56	62	62	0.132
PPO-GNN	73	57	62	61	0.305
SAC	−13	35	73*	40	0.135
Residual RL	87	68	73*	73	0.223
LLM-Policy-C	77	54	58	60	8.15

- **MSSP-I** is told the demand path over its horizon; informed RL, only today's mean.
- The best *learned* controller (∗) differs by regime: DAgger-B, PPO-Transformer, then SAC and Residual RL (both 73%).
- SAC is *negative* on stationary demand on average.
- Informed echelon heuristic: 72%, within 3 points of the best learned policy at a quarter of its mean latency.

Paper, Table 4; s/ep = mean online seconds per episode. -I informed, -B blind; unsuffixed learned agents are informed.

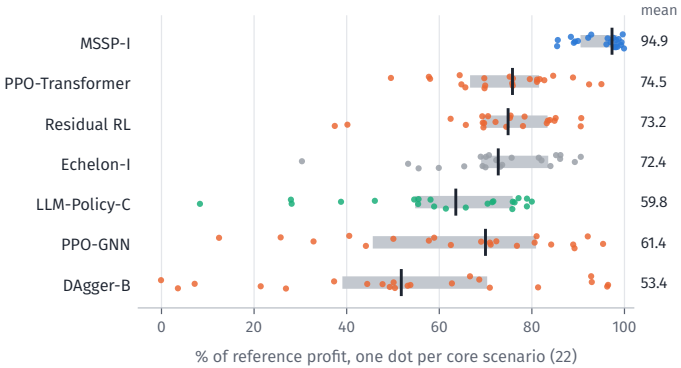
Rankings change with the regime



Paper, Table 4 (regime means).

- Below **MSSP-I**, the order changes with the regime.
- **DAgger-B** (blind imitation): 95% stationary, 33% non-stationary.
- **Blind MSSP** drops from 92% to about 60% once demand moves.
- **PPO-Transformer** has the flattest learned profile (74 / 77 / 71).
- Goodwill caps everyone below 90%; even MSSP-I falls to 89%.

Behind each average: a wide spread



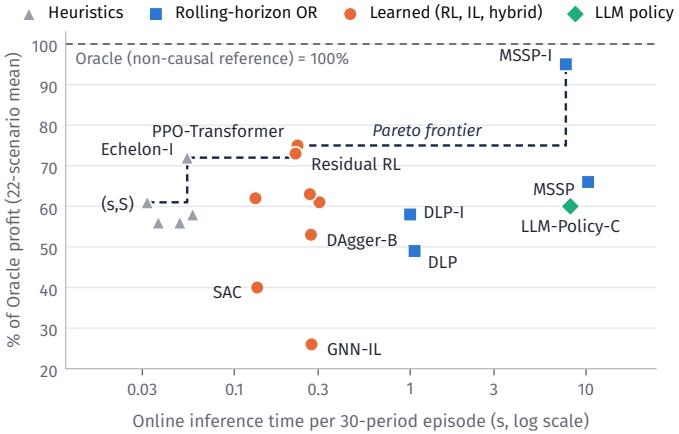
Each dot is one of the 22 scenarios; the grey bar is the middle half.

MSSP-I stays within 90.5–98.4% for the middle half. The others' middle halves span about 14 points or more; DAgger-B and PPO-GNN range from about zero to about 95%.

A single leaderboard number would hide this.

Recomputed from the released results CSV. Ranges over chosen scenarios, not confidence intervals.

Speed vs. quality: no family dominates



The Pareto frontier has a member from four families: heuristics, hybrid, deep RL and OR.

Which controller to use depends on the latency and information budget.

Paper, Table 4: 22 core scenarios. Frontier computed over causal controllers (Oracle excluded).

When does a learned policy pay for itself?

Effective per-episode cost of a policy reused over N episodes:

$$C_{\text{eff}}(N) = \frac{C_{\text{train}}}{N} + C_{\text{infer}}$$

Against MSSP-I (7.68 s/episode), PPO-Transformer (0.229 s) breaks even after

$$N^{*} = \frac{C_{\text{train}}}{7.68 - 0.229} \text{ episodes.}$$

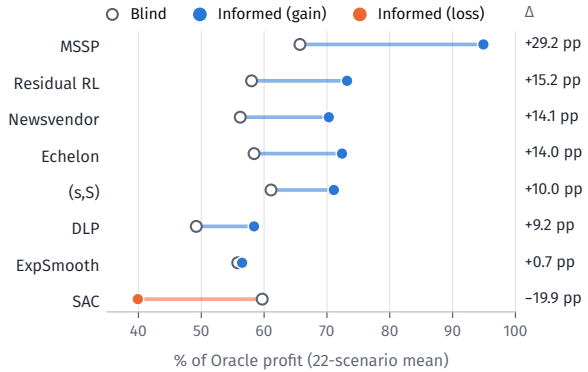
Illustration only: one GPU-hour of training would break even after about 480 episodes. We did not record exact training times.

Reading

- One-off planning favours solvers and heuristics.
- Repeated deployment across stores, SKUs and planning cycles favours reusable generalist policies.
- Against Echelon-I (72%, 0.05 s, no training) it never pays back; the case rests on a 3-point gain.

Quality still differs (95% vs. 75%), so this is a cost argument, not a dominance claim.

Information helps methods that can use it structurally

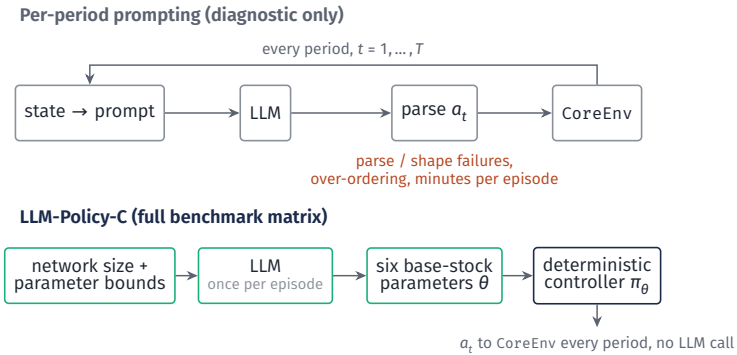


- MSSP +29.2 pp: its scenario tree is centred on the right process.
- DLP only +9.2 pp: an expected-value plan ignores variance.
- Variance-aware heuristics +10 to +14; ExpSmooth +0.7, as its informed variant gets only the variance.
- SAC **−19.9**: separately trained checkpoints; the loss is almost all on the serial chain (−36 vs. −4).

Caveat: solvers and heuristics look ahead over their horizon; informed RL sees only the current $\mu(t)$.

Paper, Table 5: paired blind / informed variants, 22 core scenarios.

Called once per episode, a small LLM sets six constants



Qwen2.5-1.5B-Instruct, run locally. LLM-Policy-C: **60%** of Oracle at 8.2 s per episode (9.0 s over all 26 rows). Its prompt holds only the network size and the allowed parameter ranges; we have not yet isolated what the LLM adds over the default settings.

Topology: profit carries over, service does not

Within the two training topologies

- PPO-GNN: 80.4% on the divergent network, 42.3% on the serial chain.
- PPO-Transformer: 78.9% vs. 70.1%; MSSP-I flat (94.4 / 95.4).
- Separate checkpoints per topology, so this is not a transfer test.

Zero-shot transfer (separate diagnostic)

- An appendix experiment, outside the 22-scenario matrix.
- A residual GCN-pool policy trained on the 9-node graph keeps positive profit on four unseen graphs: 41–84% of the newsvendor heuristic.
- But service on the new retail edges collapses. We attribute it to action decoding; the exact cause is not yet isolated.

A profit-only leaderboard would have **hidden this failure**.

Part 4

Lessons and open problems

1 The problem

2 The benchmark

3 The experiments

4 Lessons and open problems

Lessons for benchmarking sequential decision-making

1. Treat **information access** as an experimental factor. Report blind and informed variants side by side.
2. Keep architecture and information apart, or a model that sees more will look like a better model.
3. Report **latency next to quality**, and say whether training cost is amortized.
4. Prefer regime-level results to one average; leaders change across regimes.
5. Log requested *and* executed actions, so infeasible behaviour is visible.
6. State whether learned agents are generalists or specialists; it changes what a learned number means.

Trade-offs we accepted

Choice	What it bought	What it cost
OR-Gym network, 30-period episodes	comparability with Hubbs and Perez	one stylized network; end-of-horizon effects
One generalist per architecture × topology, one training run	a handful of models instead of one per scenario	no training-run variance or tuning sweeps
MSSP: 10 paths, branching over 3 periods, horizon 21 / 13, 30 s solver cap	about 8–10 s per episode	sampling error and a truncated lookahead
1.5B LLM, 4-bit, called once per episode	8 s per episode; per-period prompting took about 7 min	a small model; its value over the defaults not isolated
Python + NumPy; 22 scenarios × 10 seeds	easy to audit; 7,670 released episodes	no large-scale sweep; wide per-scenario spread
Not released: DAgger and transfer code, decentralized (MARL) evaluation, figure scripts	—	cannot be reproduced yet

Limitations

- **Scope.** 22 curated scenarios, one product, two stylized networks: controlled comparisons, not a representative sample of supply chains.
- **External data.** M5 rows replay terminal demand only; no real upstream network, prices or calendar effects.
- **Reference.** Under goodwill, the Oracle is a clairvoyant simulation-optimization benchmark, not a certified upper bound.
- **Roles differ by design.** Oracle is non-causal, MSSP-I is informed, LLM-Policy-C acts once per episode. The comparison is of deployment roles, not of one identical problem.
- **Reproducibility gaps.** DAgger training and the transfer experiment are not yet in the release; the MARL-labelled rows rerun the centralized setting.

Open problems (and possible projects)

- **Topology transfer**: find why service to new edges collapses; test edge-conditioned heads and pointer mechanisms.
- **Hybrid OR + learning**: stochastic programs producing targets for learned policies; learned warm-starts for DLP/MSSP.
- **LLM controllers**: constrained decoding, tool-verified JSON schemas, retrieval over inventory histories.
- **Richer physics**: stochastic lead times, random yields, disruptions, multi-product substitution, 100–365 period horizons.
- **Better statistics**: multi-seed training, bootstrap intervals, paired non-parametric tests, multi-objective rewards.
- **Speed**: a JAX-native backend for large-scale rollouts.

If any of these overlaps with your work, I would be glad to talk after the seminar.

Thank you

- Rankings of inventory controllers depend on information access, demand shift, topology and representation. gym-invmgmt makes those conditions explicit.
- MSSP-I is the quality reference (95%); PPO-Transformer and Residual RL give 73–75% at about 0.2 s per episode.
- Imitation was fragile under demand shift; the 1.5B LLM ran reliably only when reduced to setting six constants once per episode.

Joint work with Qinmin Vivian Hu.

code, results, weights

try it in Colab

```
pip install gym-invmgmt
```

github.com/r2barati/gym-invmgmt-paper
arXiv:2605.11355

r2barati@torontomu.ca